

Entrega 2 — Observabilidade: fundamentação dos painéis

Disciplina: Infraestrutura Cloud — UniAmérica **Integrantes:** Luis Felipe Coradi · Rubens Prado · Guilherme Royer
Repositório: <https://github.com/LuisCoradi/aula-uniamerica-infraestrutura-cloud> **Aplicação:** <https://app.trabaidafacul.site> · **API:** <https://api.trabaidafacul.site> **Ferramenta de painéis:** Grafana Cloud (acesso autenticado; demonstrado na apresentação)

1. O que mudou em relação à primeira entrega

A infraestrutura é a mesma: domínio próprio, HTTPS, proxy reverso, front-end redundante em duas origens, back-end sem endereço público e banco sem acesso de rede. O que foi acrescentado:

- **Instrumentação dos Workers.** Um módulo único (`infra/shared/log.js`) que monta cada evento como uma linha JSON e o envia. Usado pelo proxy e pelo back-end.
- **Worker novo** `trabaidafacul-monitor`. Uma sonda com Cron Trigger que checa a aplicação pelo domínio público a cada minuto e registra o resultado.
- **Correções trazidas da entrega 1 para o repositório.** O código publicado ainda tinha o tratamento antigo do HTTP 304 e o timeout que cortava o corpo da resposta; ambos já estavam corrigidos em produção e agora estão versionados.

Nada foi alterado na aplicação de tarefas em si.

2. Escolha da ferramenta

Grafana Cloud (plano gratuito), recebendo logs via API do Loki.

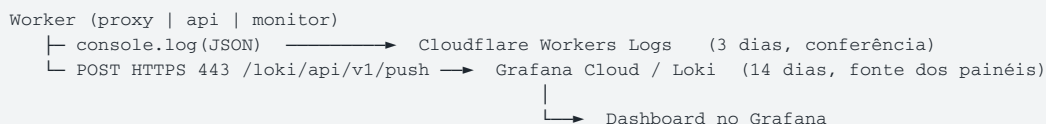
Três motivos, na ordem que pesaram:

1. **A definição do painel é exportável como arquivo.** O enunciado pede as definições dos painéis no repositório. O Grafana exporta o dashboard inteiro em JSON por um botão, e esse arquivo é reimportável — está em `observabilidade/dashboard-trabaidafacul.json`.
2. **O caminho do dado fica explícito.** Componente que gera → coleta e encaminhamento → armazenamento e consulta → painel são quatro peças distintas e demonstráveis. Se tudo ficasse dentro da Cloudflare, o caminho viraria “o provedor faz tudo”, difícil de comprovar.
3. **Retenção de 14 dias** no plano gratuito, contra 3 dias do Workers Logs da Cloudflare. Cobre a entrega e a apresentação com folga.

O Workers Logs da Cloudflare continua ligado (`observability.enabled = true`) e é usado como **fonte de conferência**: a mesma linha JSON aparece nos dois lados, o que permite provar que o painel não inventa dado.

Alternativas descartadas: **Logpush** da Cloudflare (exige plano pago), **Health Checks** gerenciados da Cloudflare (exigem plano Pro), **Better Stack** (3 dias de retenção) e **Axiom** (exportar a definição do painel só via API REST).

3. Caminho do dado



- **Coleta:** os eventos de uma invocação são acumulados em memória e enviados em **um único POST** dentro de `ctx.waitUntil()`, ou seja, depois que a resposta já foi devolvida ao usuário. Log não atrasa requisição.
- **Autenticação:** Basic Auth com um token de Access Policy limitado ao escopo `logs:write`. O token não lê nada e está guardado como `wrangler secret`, fora do repositório.
- **Rótulos do stream** (baixa cardinalidade): `aplicacao`, `servico`, `ambiente`. Os campos de alta cardinalidade (`req_id`, `rota`, `status`) vão dentro da linha JSON e são extraídos na consulta com o parser `| json`.
- **Se as credenciais não estiverem configuradas**, o envio é ignorado em silêncio e a aplicação continua funcionando normalmente.

O detalhamento dos campos está em `observabilidade/formato-do-log.md`.

4. Fundamentação dos painéis

Sete painéis. Todos alimentados por dados reais do ambiente, nenhum com dado fictício. Os valores citados como “observado” foram lidos do próprio dashboard em 17 e 18 de setembro de 2026.

Convenção das consultas: `$__interval` é o intervalo de agrupamento que o Grafana calcula a partir da janela escolhida; `$__range` é a janela inteira. Os eventos são gravados em UTC e exibidos no fuso do navegador (America/Sao_Paulo).

Painel 1 — Disponibilidade pelo domínio

Pergunta operacional. A aplicação está acessível pelo domínio configurado, agora e no período consultado?

Motivo da escolha. É o indicador que responde à pergunta mais básica da entrega 1: o acesso ocorre pelo domínio, e não pelo endereço do provedor. Como o Health Check gerenciado da Cloudflare exige plano pago, a checagem foi implementada como Worker próprio — o que tem a vantagem de ficar versionada no repositório.

Origem dos dados. Worker `trabaidafacul-monitor`, evento `checagem_disponibilidade`. Ele roda por Cron Trigger a cada minuto e faz uma requisição HTTPS para `https://app.trabaidafacul.site/` e outra para `https://api.trabaidafacul.site/health`. Campos usados: `ok` (booleano), `alvo`, `status`, `duracao_ms`.

Consulta e cálculo. Proporção entre checagens bem-sucedidas e total de checagens:

```
sum(count_over_time({aplicacao="trabaidafacul", servico="monitor"} | json | evento="checagem_disponibilidade" | ok="true" [$__range]))
/
sum(count_over_time({aplicacao="trabaidafacul", servico="monitor"} | json | evento="checagem_disponibilidade" [$__range]))
```

Numerador: checagens com `ok=true`. Denominador: todas as checagens do período. Unidade: proporção, exibida como percentual.

Recorte temporal. Janela padrão de 24 horas, atualização a cada 30 segundos, uma amostra por minuto por alvo (1.440 amostras por alvo por dia). Fuso do navegador.

Forma de visualização. Indicador numérico único (*stat*), com faixas de cor: verde a partir de 99%, laranja a partir de 95%, vermelho abaixo disso. Um número grande responde à pergunta “está no ar?” sem exigir leitura de gráfico.

Interpretação. 100% significa que todas as checagens do período responderam como esperado. Qualquer valor abaixo de 100% significa que houve minutos em que a aplicação não respondeu pelo domínio — o painel 1b diz qual alvo falhou.

Crítérios de atenção. Abaixo de 99% em 24 horas equivale a mais de 14 minutos de indisponibilidade no dia, o que já é relevante para uma aplicação simples como esta. Abaixo de 95% indica problema em curso. Esses limites foram escolhidos por convenção de mercado, não por histórico próprio — o ambiente tem poucos dias de operação, e isso é uma limitação honesta do critério.

Ação decorrente. Queda no indicador: abrir o painel 1b para saber se o alvo que falhou é o front ou a API; se for o front, olhar o painel 6 para ver se o failover entrou; se for a API, olhar os painéis 3 e 5.

Validação e limitações. Observado em 18/09/2026: 100% em 24 horas, com as duas séries do painel 1b presentes em todos os minutos. A checagem parte de dentro da rede da Cloudflare: ela prova que o domínio responde e que o caminho interno está de pé, mas **não mede a experiência de rede de um usuário em casa**. Também não detecta erro de renderização no navegador — só que o HTTP respondeu.

Painel 1b — Checagens por alvo (sucesso × falha)

Pergunta operacional. Quando a disponibilidade cai, o problema é no front-end ou na API?

Motivo da escolha. Sem essa separação, o painel 1 diz que algo caiu mas não onde. Ele também cumpre uma segunda função, explicada no item de interpretação: distinguir “não houve uso” de “não houve coleta”.

Origem dos dados. O mesmo evento `checagem_disponibilidade`, agrupado por `alvo` (`front` ou `api`) e separado por `ok`.

Consulta e cálculo.

```
sum by (alvo) (count_over_time({aplicacao="trabaidafacul", servico="monitor"} | json | evento="checagem_disponibilidade" | ok="true" [$__interval]))
sum by (alvo) (count_over_time({aplicacao="trabaidafacul", servico="monitor"} | json | evento="checagem_disponibilidade" | ok="false" [$__interval]))
```

Contagem simples de eventos por intervalo. Unidade: número de checagens.

Recorte temporal. Mesma janela do painel 1.

Forma de visualização. Série temporal com legenda em tabela, mostrando último valor e máximo. Duas linhas por alvo (ok e falha) permitem ver a transição no tempo, coisa que um número único não mostra.

Interpretação. Linhas “ok” contínuas e linhas “FALHA” no zero é o comportamento esperado. Um pico na linha de falha marca o minuto exato do problema.

Crítérios de atenção. Qualquer ponto na série de falha merece investigação. Duas falhas consecutivas no mesmo alvo indicam problema real, e não oscilação de rede.

Ação decorrente. Falha em `front`: verificar o painel 6 (redundância) e o estado das origens no Pages. Falha em `api`: verificar o painel 5 (banco) e o painel 3 (erros).

Validação e limitações. Observado: as duas séries “ok” presentes de forma contínua; nenhuma falha registrada no período. A granularidade é de um minuto — uma indisponibilidade de 20 segundos pode passar despercebida entre duas checagens.

Painel 2 — Latência por rota (p95 e mediana)

Pergunta operacional. Quais rotas estão demorando mais, e em que horários?

Motivo da escolha. É o indicador que mostra degradação antes de virar erro. Uma rota que sobe de 100 ms para 400 ms ainda responde 200, mas o usuário sente.

Origem dos dados. Worker `trabaidafacul-proxy`, evento `requisicao_api`, campo `duracao_ms` — medido entre a chegada da requisição no proxy e a resposta do back-end. Agrupado pelo campo `rota`, que é padronizado na instrumentação: `/todos/42` é registrado como `/todos/:id`, senão cada tarefa criaria uma série nova no gráfico.

Consulta e cálculo.

```
quantile_over_time(0.95, {aplicacao="trabaidafacul", servico="proxy"} | json | evento="requisicao_api" | unwrap duracao_ms [$_interval]) by (rota)
quantile_over_time(0.50, {aplicacao="trabaidafacul", servico="proxy"} | json | evento="requisicao_api" | unwrap duracao_ms [$_interval]) by (rota)
```

O p95 é o valor abaixo do qual estão 95% das requisições do intervalo; a mediana é o valor central. Unidade: milissegundos.

Recorte temporal. Janela de 24 horas, agrupamento automático conforme o zoom, atualização a cada 30 segundos.

Forma de visualização. Série temporal com uma linha por rota e legenda em tabela com último valor e máximo. Duas estatísticas na mesma escala: se p95 e mediana andam juntos, a lentidão é geral; se só o p95 sobe, o problema atinge parte das requisições — tipicamente contenção ou cache frio.

Interpretação. Observado: p95 de `/health` em torno de 140 ms (máximo 272 ms), `/todos` em torno de 103 ms (máximo 197 ms), `/todos/:id` em torno de 179 ms. São valores coerentes com uma chamada que atravessa proxy, service binding e uma consulta ao D1.

Crítérios de atenção. p95 acima de 500 ms de forma sustentada, ou p95 maior que o dobro da mediana por mais de alguns minutos. O limite de 500 ms vem do que foi observado (três a cinco vezes o normal), não de um requisito formal de desempenho — a aplicação não tem SLA definido, e essa é uma limitação do critério.

Ação decorrente. Verificar se a lentidão é do banco (painel 5) ou do caminho externo (painel 1b, duração das checagens). Se for o banco, olhar o volume de consultas; se for o caminho externo, olhar a origem servida no painel 6.

Validação e limitações. Validado com uso real da aplicação em 17/09 e 18/09, com as três rotas aparecendo no gráfico. Limitação importante: **dentro de um Worker o relógio só avança após uma operação de entrada e saída.** Como a chamada ao back-end é uma operação dessas, a medição funciona, mas é grossa: serve para comparar ordens de grandeza e detectar degradação, não para cravar milissegundos. O painel também não mede o tempo de rede entre o usuário e a borda da Cloudflare.

Painel 3 — Erros e bloqueios

Pergunta operacional. Que falhas estão ocorrendo, qual a proporção delas, e quantas são bloqueio proposital em vez de defeito?

Motivo da escolha. É o painel que separa duas coisas que um gráfico de “erros HTTP” misturaria: um 403 devolvido pela regra de acesso é a segurança funcionando; um 503 é a aplicação quebrando. Sem essa separação, um teste de bloqueio pareceria incidente.

Origem dos dados. Worker `trabaidafacul-proxy`. Dois campos: `motivo_bloqueio` (preenchido apenas quando o proxy

recusa — origem_nao_autorizada, rota_nao_permitida, metodo_nao_permitido) e status.

Consulta e cálculo.

```
sum by (motivo_bloqueio) (count_over_time({aplicacao="trabaidafacul", servico="proxy"} | json | motivo_bloqueio != "" [$_interval]))
sum(count_over_time({aplicacao="trabaidafacul", servico="proxy"} | json | status >= 500 [$_interval]))
```

A primeira consulta conta bloqueios por motivo; a segunda conta respostas de erro de servidor. Para a taxa de erro, o numerador é a contagem de 5xx e o denominador é a contagem total de respostas do proxy no mesmo intervalo. Unidade: número de respostas.

Recorte temporal. Mesma janela dos demais.

Forma de visualização. Série temporal com legenda em tabela. Cada motivo de bloqueio é uma série própria, para que o tipo de tentativa seja visível sem abrir o log.

Interpretação. Painel vazio significa que não houve nenhum erro de servidor nem nenhuma tentativa bloqueada no período — que é exatamente o estado observado em uso normal. Séries de bloqueio subindo mostram a allowlist trabalhando; a série de 5xx subindo é problema.

Crítérios de atenção. Qualquer 5xx merece investigação, porque o volume normal é zero. Bloqueios em rajada a partir de um mesmo padrão indicam varredura automatizada, caso em que a regra de rate limiting no WAF é o próximo passo.

Ação decorrente. 5xx: cruzar o `req_id` do evento com os painéis 5 e 6 para saber se foi banco ou origem. Bloqueio inesperado vindo do próprio front: revisar a allowlist de rotas, porque pode ser rota legítima fora da lista.

Validação e limitações. O painel é validado pelo cenário de bloqueio descrito na seção 5 (requisições com Origin não autorizada, método fora da lista e rota inexistente). Limitação: ele enxerga apenas o que chega ao Worker; o que o WAF da Cloudflare barra antes disso não aparece aqui, e precisa ser consultado no painel de Security da própria Cloudflare.

Painel 4 — Uso da aplicação por operação

Pergunta operacional. Quais funcionalidades são realmente usadas e como o volume varia ao longo do dia?

Motivo da escolha. Os outros painéis olham saúde; este olha comportamento. Ele responde se uma queda de tráfego é falta de uso ou falha de coleta, e mostra qual operação concentra o volume — informação que orienta onde vale otimizar.

Origem dos dados. Worker `trabaidafacul-api`, evento `operacao_negocio`, campo `operacao` com os valores `listar`, `criar`, `concluir`, `reabrir`, `excluir` e `health`. O evento é gerado dentro do back-end, depois da operação concluir, e carrega o `req_id` que veio do proxy.

Consulta e cálculo.

```
sum by (operacao) (count_over_time({aplicacao="trabaidafacul", servico="api"} | json | evento="operacao_negocio" [$_interval]))
```

Contagem de eventos por operação e por intervalo. Unidade: operações.

Recorte temporal. Mesma janela dos demais.

Forma de visualização. Série temporal por operação, legenda em tabela. A separação por operação é o que dá sentido ao painel: um total único esconderia que o volume é quase todo de leitura.

Interpretação. Observado: picos de `criar` (até 5 por intervalo) e `excluir` durante os testes, e uma linha contínua de `health`, que vem da sonda. O tráfego do monitor é marcado com `trafego: monitor` nos eventos do proxy, para não inflar a leitura de uso real.

Crítérios de atenção. Volume caindo a zero em horário de uso esperado, com o painel 1 ainda em 100%, indica que os usuários pararam de usar — não que a aplicação caiu. O contrário (volume normal com disponibilidade caindo) indica falha parcial.

Ação decorrente. Queda de volume: verificar se houve mudança no front ou se a coleta parou (painel 1b). Crescimento atípico de `criar`: verificar se não é script automatizado, cruzando com o painel 3.

Validação e limitações. Validado com uso manual da aplicação, criando, concluindo e apagando tarefas, e conferindo o aparecimento das séries correspondentes. Limitação: não há identificação de usuário na aplicação, então o painel mede operações e não pessoas.

Painel 5 — Banco de dados (D1)

Pergunta operacional. As operações do back-end com o banco estão funcionando, e com qual duração?

Motivo da escolha. Foi apontado na primeira entrega que o D1 pode ser limitado para esta arquitetura. Em vez de discutir isso no plano teórico, o painel mostra a duração real de cada tipo de consulta e permite ver se e quando ela degrada.

Origem dos dados. Worker `trabaidafacul-api`, evento `consulta_banco`, gerado por um envelope que mede o tempo em volta de cada chamada ao D1. Campos: `operacao` (`contar_tarefas`, `listar_tarefas`, `inserir_tarefa`, `alternar_tarefa`, `excluir_tarefa`), `ok` e `duracao_ms`.

Consulta e cálculo.

```
quantile_over_time(0.95, {aplicacao="trabaidafacul", servico="api"} | json | evento="consulta_banco" | unwrap duracao_ms [$_interval]) by (operacao)
sum(count_over_time({aplicacao="trabaidafacul", servico="api"} | json | evento="consulta_banco" | ok="false" [$_interval]))
```

p95 da duração por tipo de operação, e contagem de consultas que falharam. Unidade: milissegundos e número de falhas.

Recorte temporal. Mesma janela dos demais.

Forma de visualização. Série temporal por operação. Separar por tipo de operação é o que permite dizer se a escrita está mais cara que a leitura — que é a pergunta relevante para um SQLite gerenciado.

Interpretação. Observado: `contar_tarefas` em torno de 132 ms (máximo 266 ms), `excluir_tarefa` em 175 ms, `inserir_tarefa` em torno de 163 ms. Escrita custando mais que leitura é o comportamento esperado do D1. Nenhuma falha registrada.

Crítérios de atenção. Qualquer consulta com `ok=false` merece investigação imediata, porque o volume normal é zero. p95 de escrita acima de 500 ms de forma sustentada indicaria que o banco virou o gargalo — momento em que a limitação apontada pelo professor deixaria de ser teórica.

Ação decorrente. Falhas: verificar se o schema mudou ou se o limite de armazenamento do plano gratuito foi atingido. Duração subindo junto com o volume de operações no painel 4: é carga, e a discussão passa a ser de arquitetura.

Validação e limitações. Validado comparando um `req_id` específico: a mesma requisição aparece no proxy, no evento de negócio e no evento de banco. A mesma limitação de relógio do painel 2 vale aqui, e de forma mais forte, porque as durações são menores. O painel também não enxerga métricas internas do D1 (linhas lidas, tamanho do banco) — para isso existe a aba de métricas do próprio serviço na Cloudflare, que é administrada pelo provedor.

Painel 6 — Redundância do front-end

Pergunta operacional. Como os dois pontos de atendimento se comportam, e quando houve troca de origem?

Motivo da escolha. A redundância era requisito da primeira entrega, mas até aqui só existia no papel: nada mostrava que as duas origens realmente atendiam, nem quando o failover entrava. Este painel transforma o requisito em evidência.

Origem dos dados. Worker `trabaidafacul-proxy`. Evento `requisicao_front`, com os campos `origem_servida` e `failover`; e evento `origem_indisponivel`, gerado a cada tentativa de origem que falha.

Consulta e cálculo.

```
sum by (origem_servida) (count_over_time({aplicacao="trabaidafacul", servico="proxy"} | json | evento="requisicao_front" [$_interval]))
sum by (origem_servida) (count_over_time({aplicacao="trabaidafacul", servico="proxy"} | json | evento="origem_indisponivel" [$_interval]))
sum(count_over_time({aplicacao="trabaidafacul", servico="proxy"} | json | evento="requisicao_front" | failover="true" [$_interval]))
```

Contagem de requisições atendidas por origem, quedas detectadas e requisições que precisaram de failover. Unidade: requisições.

Recorte temporal. Mesma janela dos demais.

Forma de visualização. Série temporal com as duas origens e a série de quedas. O objetivo é que a troca de origem seja visível como um evento no tempo, com hora exata para servir de evidência.

Interpretação. Observado: as duas origens aparecem atendendo (`trabaidafacul.pages.dev` e `trabaidafacul-b.pages.dev`), o que confirma que a distribuição por hash de IP está em funcionamento. A série de quedas em zero

significa que nenhuma origem falhou por conta própria.

Critérios de atenção. Qualquer ponto na série de quedas indica origem com problema. Todo o tráfego concentrado numa origem só por muito tempo não é necessariamente erro — a afinidade é por IP, e com poucos visitantes é esperado que um só lado atenda.

Ação decorrente. Quedas registradas: verificar o estado daquele projeto no Cloudflare Pages e, se necessário, republicar. Failover permanente em uma das origens: tratar como incidente, porque a redundância deixou de existir.

Validação e limitações. Validado pelo cenário de falha controlada descrito na seção 5: trocando `ORIGIN_1` por um host inexistente, a série da origem 1 some, a de quedas sobe e o `X-Failover: sim` aparece na resposta. Limitação: o failover é **reativo** — o proxy descobre que a origem caiu ao tentar usá-la, porque não há verificação de saúde ativa (isso seria o Load Balancer pago da Cloudflare). Na prática, a primeira requisição após a queda é a que paga o custo da descoberta.

5. Como interpretar ausência de dados

Período sem pontos em um painel **não** significa “nenhum erro” nem “funcionamento normal”. Significa que não houve coleta naquele intervalo, e as causas possíveis são duas: não houve tráfego, ou o envio falhou.

O que separa os dois casos é o painel 1b: ele recebe uma checagem por minuto, independentemente de existir usuário. Portanto:

- **Painéis 2, 4, 5 e 6 vazios, com o 1b cheio** → não houve uso da aplicação naquele período. Comportamento esperado de madrugada.
- **Todos os painéis vazios, inclusive o 1b** → a coleta parou. Investigar o envio: `wrangler tail trabaিদafacul-proxy` mostra o evento `falha_envio_observabilidade` com o código HTTP da recusa.
- **Painel 3 vazio** → estado normal, e não falta de dado: ele só desenha quando existe erro ou bloqueio.

Cada painel enxerga apenas uma parte do sistema, e isso está explicitado nas limitações de cada um. Em resumo: os painéis medem o que a aplicação registra a partir da borda da Cloudflare para dentro. O trecho entre o usuário e a borda, e o funcionamento interno dos serviços gerenciados, não são visíveis daqui.